Uncertainty-Aware Portfolio Management With Risk-Sensitive Multiagent Network

Kidon Park, Hong-Gyu Jung, Tae-San Eom, and Seong-Whan Lee, *Fellow, IEEE*

***Abstract*— As deep neural networks (DNNs) have gained considerable attention in recent years, there have been several cases applying DNNs to portfolio management (PM). Although some researchers have experimentally demonstrated its ability to make a profit, it is still insufficient to use in real situations because existing studies have failed to answer how risky investment decisions are. Furthermore, even though the objective of PM is to maximize returns within a risk tolerance, they overlook the predictive uncertainty of DNNs in the process of risk management. To overcome these limitations, we propose a novel framework called risk-sensitive multiagent network (RSMAN), which includes risk-sensitive agents (RSAs) and a risk adaptive portfolio generator (RAPG). Standard DNNs do not understand the risks of their decision, whereas RSA can take risk-sensitive decisions by estimating market uncertainty and parameter uncertainty. Acting as a trader, this agent is trained via reinforcement learning from dynamic trading simulations to estimate the distribution of reward and via unsupervised learning to assess parameter uncertainty without labeled data. We also present an RAPG that can generate a portfolio fitting the user's risk appetite without retraining by exploiting the estimated information from the RSAs. We tested our framework on the U.S. and Korean real financial markets to demonstrate the practicality of the RSMAN.**

***Index Terms*— Deep learning, financial trading, portfolio management (PM), reinforcement learning (RL), uncertainty quantification.**

I. Introduction

PORTFOLIO denotes a collection of investment instruments, such as stocks, bonds, commodities, and other cash equivalents. It is common for a portfolio investment to be considered a stable strategy rather than investing in a single asset due to the effect of dispersed risks. Portfolio management (PM) selects the right investment in a proper proportion to make a profit along with balancing risk. Traditional PM strategies [1], [2] are built upon the fundamental strategy [3] of mean-variance optimization that maximizes returns and minimizes risk over a predefined period. These conventional approaches simply assume that portfolio's return and risk are estimated by reference to the historic return and volatility of assets. However, this assumption is violated by the dynamic characteristics of the market; thus, portfolio performance could deteriorate in the changing market.

Recently, DNN-based approaches have demonstrated considerable achievement in PM tasks. In particular, as the purpose of PM is to satisfy the demands of investors by sequentially rebalancing portfolios, reinforcement learning (RL) has been popularly used [4]–[6]. Previous works usually aimed at learning risk-return balanced PM strategies based on risk-sensitive RL [7], which focuses on reducing the variance of the cumulative reward to manage portfolio risk. However, this approach could lead to counterintuitive strategies in which it incurs a large loss when models have received unexpectedly large rewards in order to keep the variance small [8]. In addition, there is a need to retrain by adjusting weights to account for risks and returns when the user's risk appetite changes. In the case of real-world service, there are at least hundreds of customers with different risk appetites who expect highly personalized investment advice. Furthermore, previous works overlook the fact that DNNs take a risk decision on outlier despite their lack of awareness. This decision can lead to large losses; thus, the predictive uncertainty of DNNs must be considered.

To overcome these limitations, we focus on assessing the predictive uncertainty of DNNs from different sources and building an adaptable investment strategy depending on the user's risk appetite change by considering predictive uncertainty. We first assert that it is necessary to measure the predictive uncertainty represented by two types as follows: market uncertainty (also referred to as aleatoric uncertainty) and parameter uncertainty (also referred to as epistemic uncertainty). Market uncertainty refers to the randomness of an investment's outcome from an intrinsically stochastic market and is regarded as irreducible no matter how many data are provided. This can be described as a spread of the distribution of reward. On the contrary, parameter uncertainty, which arises from limited data, is essential to withhold investment if a given market condition is rarely observed during the training period. It is inferred to be reduced by collecting more data and can be represented as a dispersion of the distribution of the model's parameters. As the risks of each investment asset contribute to

Manuscript received 5 February 2021; revised 29 September 2021 and 3 February 2022; accepted 8 May 2022. Date of publication 23 May 2022; date of current version 5 January 2024. This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean Government (MSIT) (No. 2022-0-00984, Development of Artificial Intelligence Technology for Personalized Plug-and-Play Explanation and Verification of Explanation and No. 2019-0-00079, Artificial Intelligence Graduate School Program, Korea University). *(Corresponding author: Seong-Whan Lee.)*

Kidon Park, Hong-Gyu Jung, and Tae-San Eom are with the Department of Brain and Cognitive Engineering, Korea University, Seoul 02841, South Korea (e-mail: kd_park@korea.ac.kr; hkjung00@korea.ac.kr; ts_eom@korea.ac.kr).

Seong-Whan Lee is with the Department of Artificial Intelligence, Korea University, Seoul 02841, South Korea (e-mail: sw.lee@korea.ac.kr).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TNNLS.2022.3174642.

Digital Object Identifier 10.1109/TNNLS.2022.3174642

2162-237X © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.

the portfolio risk, if the model understands the uncertainty of its own decision, generating a portfolio considering the user's risk appetite without retraining is possible.

To consider the two types of uncertainty for PM tasks, we propose a novel framework called risk-sensitive multiagent network (RSMAN), comprising risk-sensitive agents (RSAs) and a risk adaptive portfolio generator (RAPG). RSA can estimate both uncertainties with a two-branch structure. The aleatoric branch is trained to estimate the reward distribution obtained from the dynamic trading simulation. To estimate the reward in the form of distribution, we utilize quantile regression (QR) [9], which is capable of constructing the entire conditional distribution of the target variable. From the estimated distribution, we can obtain both long/short positions and market uncertainty. The epistemic branch is trained to predict parameter uncertainty via learning two mapping functions based on unsupervised learning, which describes the abnormality of the given market conditions by calculating the distance between two projected points without an outlier label; it is similar to one-class classification [10]. Finally, we design RAPG to determine a portfolio proportional to each asset's investment score, which is designed based on commonsense, by exploiting the information collected from the RSAs. Consequently, it can generate a risk-adapted portfolio by reducing the proportion of investments in both highly uncertain assets.

To summarize, our contributions are given as follows.

1) We propose a novel reward definition appropriate for the investment domain.
2) To the best of our knowledge, this is the first attempt to explicitly assess both types of uncertainty in DNNs for the PM task.
3) We propose a novel portfolio generator called RAPG that mimics rational investors, generating a portfolio suitable for the user's risk appetite by considering the two types of uncertainty without retraining.
4) We show the validity of our framework with extensive experiments in real stock markets such as the NYSE, NASDAQ, and KOSPI.

The remainder of this article is organized as follows. Related works are introduced in Section II. In Section III, we present the basic concepts of RL and QR, and financial terms and formulate the PM problem. The details of the RSMAN are illustrated in detail in Section IV. Extensive experiments of RSMAN on benchmark datasets collected from diverse real financial markets are presented in Section V. Finally, we provide concluding statements and the scope of future works in Section VI.

II. Related Works

A. Machine Learning-Based Portfolio Management

A few decades ago, research was being conducted on predicting financial signals built upon supervised learning (SL) or RL. Being able to consider an investor risk tolerance or transaction costs, RL has been preferred in the financial domain over SL. A pioneering work [11] adopts Q-learning [12] to make better investment decisions, and further strategies based on deep RL have been recently announced with the development of denoising skills [6], [13], efficient representation learning [14]–[16], and data augmentation [17].

Meanwhile, PM based on deep RL can be categorized into a single-agent approach [17], [18] and a collaborative multiagent approach [4], [19]. The single-agent approach has a limitation in that efficient learning is difficult because the action space increases rapidly as the number of assets constituting the portfolio increase. For this reason, the experimenter chooses a few assets manually, and the portfolio performance could be biased depending on the selected assets. In addition, sufficiently dispersing the risk is difficult due to the fewer assets.

In contrast, previous collaborative multiagent methods have demonstrated that it operates effectively even with many assets because efficiently learning the representation suitable for a specific asset is possible by having a designated agent for each asset. Simply put, the general flow of this approach is given as follows. In the first stage, the agents for individual assets predict a fluctuation of price to obtain a long or short signal. Subsequently, in the second stage, capital is allocated according to the degree of fluctuation of the predicted prices to achieve high profits. We follow this approach due to the advantages of this approach.

Finally, a few recent works [4], [20], [21] have attempted to manage not only profitability but also risk. A commonly used approach is to utilize the risk-sensitive RL [22] to reduce the variance of the cumulative reward by maximizing risk-adjusted utility functions such as the Sharpe ratio [23] and Calmar ratio [24]. However, this approach prevents investment opportunities that can generate higher profits than usual to lower the variance of the cumulative reward. In contrast, some studies [20], [21] proposed a novel risk-averse objective function by measuring the uncertainty of step-wise rewards to solve the problems of traditional risk-sensitive RL methods. However, these approaches cannot assess the uncertainty of the model's decision and cannot change the portfolio corresponding to the user's appetite to risk after training.

B. Uncertainty Quantification

Recently, various approaches have been proposed to measure the predictive uncertainty of DNNs [25]–[27], and the validity of these approaches has been proven for few examples such as the MNIST dataset [28]. A promising framework for estimating the uncertainty associated with DNNs' predictions is provided by the Bayesian neural network (BNN) [26], [29]. BNN is built by introducing stochastic weights into the neural network to estimate the distribution of the target variable. However, BNN requires prohibitive computational cost, significant modifications to the standard training procedure, and repeated sampling for assessing predictive uncertainty. On the other hand, the frequent approaches require learning multiple networks [27] or taking several samples with dropout during the test time [25] as predictive uncertainty is estimated through the variation of the outputs. However, these approaches are unsuitable for real-time trading as the portfolio comprises numerous assets and requires individual networks for each asset, resulting in a computational burden that increases linearly with the number of assets. For this problem, we introduce

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.

RSA that can assess the uncertainty with only one forward pass suitable for PM the task in Section IV.

III. Background

Here, we present the basic concepts of RL with Q-learning, QR, and financial terms for better understanding.

A. Reinforcement Learning

RL is a realm of machine learning that teaches an agent defined within a specific environment to select an optimal action from the action space to maximize the cumulative reward. The RL problem is usually formulated as Markov decision processes, which is defined as a tuple $(\mathcal{S}, \mathcal{A}, R, P, \gamma)$, where $\mathcal{S}$ refers to the state space, $\mathcal{A}$ is the action space, R is the reward function, P is the state transition probability represented as $P(s'|s, a)$, and $\gamma \in [0, 1]$ is the discount factor. The policy of an agent is indicated by $\pi(a|s)$ that is a distribution over actions when states are given. To assess the quality of a policy π, the action-value function Q_π is defined as

$$Q_\pi(s_t, a_t) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \middle| s_t, a_t \right] \tag{1}$$

where r_t is the reward at time t. The goal of RL is to find the optimal policy π^*, that is, $Q_{\pi^*}(s, a) = \max_\pi Q_\pi(s, a)$.

To obtain this, Q-learning approximates the optimal action-value function $Q_{\pi^*}(s, a)$ with a function approximator such as DNNs. After learning the action-value function, optimal policy can be derived by performing the action that provides the best action value in each state, i.e., $a = \arg\max_{a' \in \mathcal{A}} Q(s, a')$.

B. Quantile Regression

We introduce QR that is the basis for estimating market uncertainty. QR is an extension of the ordinary least squares (OLS) regression. Given a dataset $D = \{(x_n, y_n)\}_{n=1}^N$, OLS regression obtains the relation between a target variable Y and a predictor variable X by minimizing the sum of the squares of an error $\sum_{n=1}^N (y_n - f_\theta(x_n))^2$, where $f_\theta(\cdot)$ indicates the function approximator parameterized by θ. However, OLS regression focuses only on the mean of the conditional distribution of the target variable, and thus, it cannot express a prediction interval or uncertainty.

In contrast to OLS regression, QR can provide richer details regarding the distribution of the target variable by estimating each quantile for $\tau \in (0, 1)$. The τth quantile refers to the value dividing a probability distribution into two regions where the cumulative probability of the left region is τ, and the opposite is $1 - \tau$. For example, the 100-quantiles are called percentiles and are a specialized form of quantiles, which splits a distribution into hundred regions.

The τth quantile of the distribution of the target variable Y when given x_n is expressed as follows:

$$Q_Y(\tau|x_n) := \inf\{y \in \mathbb{R} : F_Y(y|x_n) \geq \tau\} \tag{2}$$

where $F_Y(y|x_n)$ refers to the conditional cumulative distribution of the target variable. The purpose of QR is to approximate the quantile $Q_Y(\tau|x_n) \approx f_{\theta_\tau}(x_n)$ with the parameter θ_τ of the estimator in terms of τ. An estimator can obtain the τth quantile by minimizing $\sum_{n=1}^N \rho_\tau(y_n - f_{\theta_\tau}(x_n))$, where $\rho_\tau(u)$ denotes the asymmetric absolute loss function defined as $\rho_\tau(u) = u \times (\tau - \mathbb{I}_{u<0})$, u is the error, and $\mathbb{I}$ is the indicator function. Unlike the mean-squared loss used in OLS regression, the asymmetric absolute loss function gives a weight of τ for positive error and $\tau - 1$ for negative error to obtain the quantile. With this loss function, a higher quantile is estimated for higher τ, while the lower quantile is estimated for lower τ.

If one wants to know about the prediction interval of the target variable, it can be computed by combining two quantile regressors. For example, the 95% prediction interval is calculated as the difference between $f_{\theta_{\tau=0.025}}(x)$ and $f_{\theta_{\tau=0.975}}(x)$.

C. PM Problem Formulation

We assume that a financial market environment is discrete time with a finite-horizon time and sufficiently liquid such that each trade can be executed immediately without any market effect. Then, we use a collection of $m + 1$ assets $M = \{0, 1, 2, \ldots, m\}$. Here, M_0 is the risk-free asset representing a bank deposit with an annual interest rate of 3% and the rest is m risk assets.

Let the price information of each risk asset be designated as a stock price vector $\mathbf{p}^{(i)} = (p_1^{(i)}, p_2^{(i)}, \ldots, p_t^{(i)}, \ldots, p_T^{(i)})^\top$, where $p_t^{(i)}$ denotes the closing price of the ith asset at time step t and T denotes the total number of trading days. For $t \geq l$, the market condition of the ith asset at time t is summarized as the state $\mathbf{s}_t^{(i)} \in \mathcal{S}$ composed of a time window vector

$$\mathbf{s}_t^{(i)} = \left(\frac{p_{t-l+1}^{(i)} - p_t^{(i)}}{p_t^{(i)}}, \frac{p_{t-l+2}^{(i)} - p_t^{(i)}}{p_t^{(i)}}, \ldots, \frac{p_{t-1}^{(i)} - p_t^{(i)}}{p_t^{(i)}} \right)^\top \tag{3}$$

where l is the window length. Here, we use price change rates instead of absolute prices as input signals to change the price in the datasets to a common scale.

Now, the PM task includes positioning and allocation. First, we introduce notations for positioning that is used to predict the direction of price changes. The position space $\mathcal{A}$ expressed as the real-time trading position $a_t^{(i)} \in \mathcal{A} := \{\text{long}, \text{short}\} = \{1, -1\}$ is determined for each time step t. The term "long position" refers to buying an asset at time t and then selling it when the position is changed to short. On the contrary, "short position" indicates that an agent borrows an asset and sells it at time t and buys it back when the position is changed to long. The long position makes profits when an asset price rises, while the short position makes profits when an asset price falls. To this end, we optimize $\pi(a_t^{(i)}|s_t^{(i)})$ as a mapping function from the state space $\mathcal{S}$ to the position space $\mathcal{A}$.

Second, we present the notations for asset allocation. The asset allocation procedure determines the percentage to be invested in each asset. A portfolio at time t is represented by a vector $\mathbf{b}_t = (b_t^{(0)}, b_t^{(1)}, \ldots, b_t^{(m)})^\top$ with the ratio $b_t^{(i)}$ of the investment on the ith asset. The sum of all the portfolio

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.

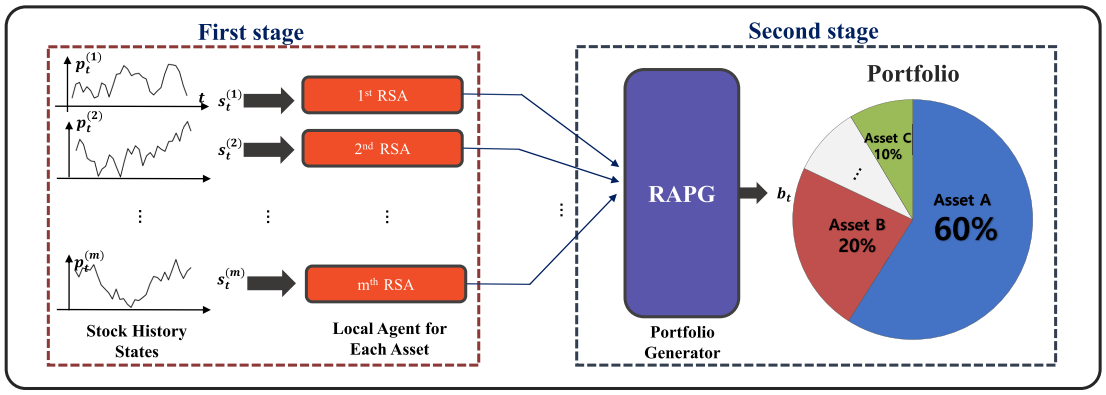


Fig. 1. Overview of our framework with a concrete example. In the first stage, each RSA receives preprocessed data to estimate positions, the distribution of reward, and parameter uncertainty. Subsequently, RAPG makes a portfolio b_t.

ratios is constrained to one, that is,

$$\sum_{i=0}^{m} b_t^{(i)} = 1 \tag{4}$$

where $b_t^{(i)} \geq 0$ for all assets.

IV. RSMAN: A Risk-Sensitive Multiagent Network for Portfolio Management

As mentioned in Section II, we follow the multiagent approach to obtain the effect of dispersed risk with limited human intervention. Fig. 1 shows the overall architecture of our framework comprising RSAs and RAPG. In a nutshell, in the first stage, each RSA is independently trained to maximize the rewards obtained from dynamic trading on individual assets while estimating uncertainty when each market condition $\mathbf{s}_t^{(i)}$ is given. In the following stage, under the assumption that each RSA has learned the policy yielding an optimal reward, RAPG generates a portfolio with a high percentage of investment ratios in high-yield and low-risk assets using the information collected from RSAs.

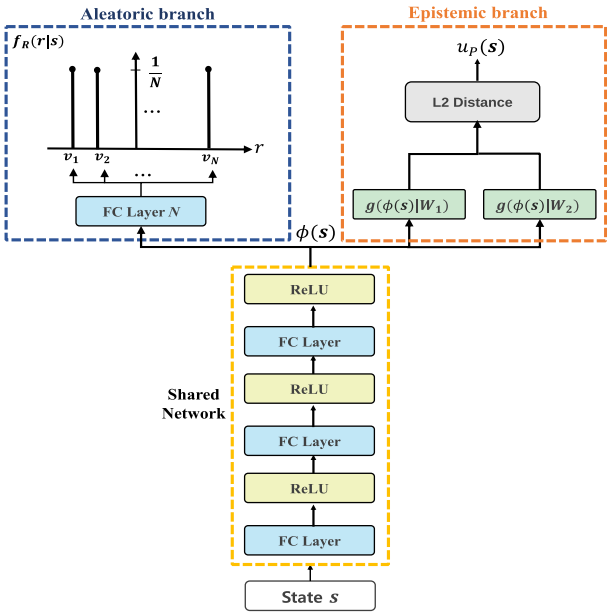


Fig. 2. Overall architecture of RSA.

A. Risk-Sensitive Agent for Each Asset

By omitting the index i for an asset, Fig. 2 shows the two-branch structure of RSA that shares parameters θ between the aleatoric and epistemic branches. We elaborate on the details of each branch as follows.

1) Aleatoric Branch for Q-Learning: The role of the aleatoric branch is to infer a reward distribution, thereby obtaining a profitable position and market uncertainty. To this end, in this study, unlike estimating the action-value function in generic Q-learning, RSA aims to learn a reward function newly defined for the trading environment.

Specifically, the purpose of general RL is to obtain a policy that maximizes the cumulative reward by estimating the optimal action value as in (1). This method is suitable for situations in which the agent must act by reflecting the rewards it can obtain in the far future, even if it makes a short-term loss; for example, the game of chess or Go. On the contrary, in the trading environment, short-term loss always has a negative impact on long-term profit. In addition, the return rate on investment made at time t is determined only by the investment position, the entry price p_t, the payoff price p_η, and the holding period $\eta - t$, where η is the payoff time defined as $\eta = \min_{\eta > t}\{\eta : a_t - a_\eta \neq 0\}$. In other words, no matter how the price changes after payoff, it does not affect the return rate on investment made at time t.

Thus, the generic action-value function reflecting the reward after payoff is not an appropriate indicator for assessing the quality of the trading policy. As a result, we define the new reward function as follows:

$$r(s_t, a_t) = a_t \times \frac{p_\eta - p_t \times (1+\mu)^{(\eta - t)}}{p_t \times (\eta - t)} \tag{5}$$

where μ is the growth rate of the market index during the training period. It is worth noting that the delayed reward in (5) is different from the immediate reward in general RL as it refers to future action and a price. Furthermore, the growth rate plays a role in preventing holding an asset for excessively long time. More specifically, if there is no consideration of

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.

the growth rate, the agent may be satisfied with the buy-and-hold strategy due to the long-term upward nature of stocks in general.

Now, we train the aleatoric branch to predict the reward based on QR as follows:

$$L_{\text{aleatoric}}(\theta) = \frac{1}{N}\sum_{n=1}^{N}\sum_{t=l}^{T}\rho_{\tau_n}(r(s_t, a_t = 1) - \upsilon_n) \tag{6}$$

where υ_n is the predicted quantile of the discretized $\tau_n = (n/N + 1)$ for $1 \leq n \leq N$. RSA predicts all quantiles with the N-dimensional output layer, as shown in the aleatoric branch of Fig. 2. Through the predicted quantiles, the reward distribution is defined as $f_R(r|s, a) = (1/N)\sum_{n=1}^{N}\delta_{\upsilon_n}$, where δ_{υ_n} is the Dirac delta function at υ_n. By adjusting the locations of quantiles, it can estimate asymmetric and multimodal distribution without any assumption about distribution shapes.

During the training period, RSA selects the action based on the sign of the random sample from the reward distribution to explore an environment as follows:

$$a_t = \text{sign}(r) \tag{7}$$

where $r \sim f_R(r_t|s_t, a_t = 1)$ and sign$(\cdot)$ is the sign function.

After training, market uncertainty is finally calculated by the standard deviation of the distribution with only a single forward pass as follows:

$$u_M(s_t) = \left[\int_{-\infty}^{\infty}(r - \bar{\upsilon})^2 f_R(r|s_t, a_t = 1)dr\right]^{1/2} \tag{8}$$

where $\bar{\upsilon}$ is the mean of the reward distribution. In this study, we only estimate the distribution of the long position $a_t = 1$ as the reward for each position is symmetric with respect to the y-axis by (5).

2) Exploration and Exploitation Tradeoff: The exploration and exploitation tradeoff is a fundamental issue in the RL research field [12], [30]. Exploration refers to the process by which an agent explores an unfamiliar environment to be sufficiently aware, and exploitation selects the most rewarding action based on experience. Determining the policy hastily without sufficient exploration can lead to local minima, and thus, an appropriate strategy is required. ϵ-greedy [12] is a straightforward method to balance exploration and exploitation, selecting a random action with probability ϵ and optimal action with probability $1 - \epsilon$. However, it uses a heuristic parameter ϵ and is not adaptable according to the learning process.

To overcome this limitation, there are a variety of efficient exploration methods based on the estimated uncertainty of the agent [31]–[36]. According to the principle of optimism in the face of uncertainty [37], [38], these methods add an exploration bonus to the values of state–action pairs, which is proportional to the uncertainty of the agent, to define whether exploration is valuable or not. For example, bootstrapped DQN [31], inspired by Thomson sampling [39], applies multihead DNNs, called Q-ensemble, to approximate the distributions of state–action values and acts greedily with respect to random samples from the distributions. Upper confidence bounds (UCB) exploration [32] adds the standard deviation of Q-ensemble to the values of state–action pairs as an exploration bonus.

We use the estimated reward distribution for exploration as in (7) during the training period, which is similar to [31]. On the other hand, for exploitation during the test period, the action is selected depending on the sign of the mean of the reward distribution as follows:

$$a_t = \text{sign}\left(\int_{-\infty}^{\infty} r f_R(r|s_t)dr\right). \tag{9}$$

3) Epistemic Branch for Parameter Uncertainty: By understanding only market uncertainty inherent in stock price data, coping with situations that have not been observed during the training procedure is impossible. For example, the stock price crash in the COVID-19 pandemic is an unprecedented situation; thus, the agent's parameters could be highly uncertain. Therefore, parameter uncertainty is an important factor in PM based on DNNs.

To estimate parameter uncertainty, we introduce the principle idea of outlier detection based on unsupervised learning, which works without labeled data. In the context of outlier detection, an outlier is defined as an observation that deviates from the dense regions of the data. As the parameter estimates of the model are uncertain in sparsely populated regions of the data, an outlier detection method is a reasonable way to estimate parameter uncertainty. In addition, there is no clear criterion for classifying abnormal market conditions; thus, it is a suitable approach for PM tasks.

The representative research of this approach is deep support vector data description (deep SVDD) [10]. The principle is to train a mapping function $g(\cdot; \mathbf{W}) := \mathcal{S} \to \mathcal{F} \subseteq \mathbb{R}^h$ to map $\mathbf{s} \in \mathcal{S}$ to near a specified center $\mathbf{c} \in \mathcal{F}$ under the assumption that the majority of training data comprises inliers. The mapping function $g(\cdot; \mathbf{W})$ is trained by minimizing loss as follows:

$$L(\mathbf{W}) = \frac{1}{T}\sum_{t=1}^{T}\|g(\mathbf{s}_t; \mathbf{W}) - \mathbf{c}\|^2. \tag{10}$$

Assuming that the training sample consists mostly of inliers, minimizing the loss allows the inliers to be well mapped close to $\mathbf{c}$ by extracting high-level representations commonly shared by inliers. On the other hand, for the samples from the sparse regions of a dataset, the function will map the samples far from center $\mathbf{c}$ because the function is not sufficiently learned on the regions. With this mapping function, the parameter uncertainty $u_P(\mathbf{s}_t)$ is measured by the distance between the specified center $\mathbf{c}$ and the mapped point $g(\mathbf{s}_t; W)$ as follows:

$$u_P(\mathbf{s}_t) = \|g(\mathbf{s}_t; \mathbf{W}) - \mathbf{c}\|^2. \tag{11}$$

Unlike MC dropout [25] and BNN [26], this approach is more computationally efficient way because parameter uncertainty can be measured with only a single forward pass.

However, the center $\mathbf{c}$ must be carefully chosen by the experimenter before training to avoid a trivial solution that indicates a constant function mapping all samples to the same point. To learn nontrivial solutions without a tricky selection of the center point, we propose the mutual distance method

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.

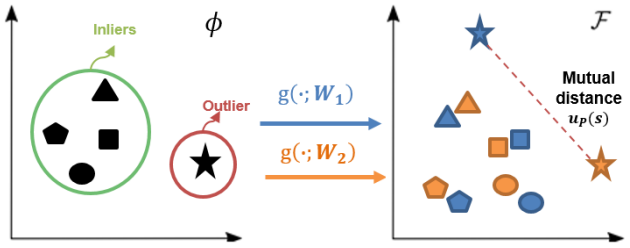


Fig. 3. Simple description of the mutual distance metric for parameter uncertainty. Each shape represents high-level representation of each data and the blue and orange shapes are the mapped points by $g(\cdot; \mathbf{W}_1)$ and $g(\cdot; \mathbf{W}_2)$, respectively.

that measures the distance between two points projected by two mapping functions. Specifically, as marked in Fig. 2, let $\phi(\mathbf{s}) \in \Phi \subseteq \mathbb{R}^d$ be a high-level representation extracted from the shared network. Then, we define a mapping function $g(\cdot; \mathbf{W}) := \Phi \to \mathcal{F} \subseteq \mathbb{R}^h$, where $\mathbf{W}$ is an $h \times d$ weight matrix and $\mathcal{F}$ denotes the output space. The two mapping functions of the epistemic branch $g(\cdot; \mathbf{W}_1)$ and $g(\cdot; \mathbf{W}_2)$ interactively determine the center point, which is adjusted as learning progresses. In this study, we simply define the two functions as matrix multiplication, that is, $g(\mathbf{s}; \mathbf{W}) := \mathbf{W}\phi^\top(\mathbf{s})$. Finally, the loss function for the epistemic branch is given by

$$L_{\text{epistemic}}(\theta) = \sum_{t=l}^{T} \left\| \mathbf{W}_1 \phi^\top(\mathbf{s}_t) - \mathbf{W}_2 \phi^\top(\mathbf{s}_t) \right\|_2^2 \tag{12}$$

where $\|\cdot\|_2^2$ is the L2 distance.

Consequently, the parameter uncertainty is measured by the distance between the two mapped points

$$u_P(s_t) = \left\| \mathbf{W}_1 \phi^\top(\mathbf{s}_t) - \mathbf{W}_2 \phi^\top(\mathbf{s}_t) \right\|_2^2. \tag{13}$$

Fig. 3 describes an example of how parameter uncertainty quantification based on the mutual distance metric works. In the case of inliers, data are mapped to a near point by the two functions, whereas in the case of outliers, data are mapped to a far distance, resulting in high parameter uncertainty.

4) Total Loss of RSA: Finally, the total loss of RSA is defined as the sum of each branch's loss as follows:

$$L(\theta) = \alpha L_{\text{aleatoric}}(\theta) + \beta L_{\text{epistemic}}(\theta) \tag{14}$$

where α and β are weights for the aleatoric and epistemic loss, respectively. Through the total loss, we train RSA with the stochastic gradient descent for each asset $1 \le i \le m$. As mentioned in [40] and [41], the advantages of the shared network between the two branches are that the dual loss regularizes the network to extract a common representation that supports multiple tasks and brings some computational advantages. Furthermore, if each branch is trained independently as an individual model without sharing information with each other, the epistemic branch does not properly estimate the parameter uncertainty of the information measured in the aleatoric branch. Thus, the epistemic branch should be aware of which parameters are being used in the other branch by using the shared network.

B. Risk Adaptive Portfolio Generator

Here, we introduce RAPG that utilizes the estimated uncertainties to manage risk. As market uncertainty and parameter uncertainty usually depend on different scales, dealing with the two uncertainties is a challenging issue. To this end, we design RAPG according to the two intuitions that are common in the financial domain.

1) While the absolute value of uncertainty can be meaningful, the relative trend is more crucial. For example, when uncertainty rises steeply compared to the recent market, investors reduce investment in risky assets in order to hedge the risk; otherwise, even though the absolute value of uncertainty is still high, they begin buying risky assets when the relative trend turns downward.
2) A reasonable investor invests when the expected profit rate exceeds risks [42]. The purpose of investment is to gain profit while enduring the riskiness of the asset; thus, if the risk exceeds the expected profit rate, investors would not make an investment decision.

Following the first intuition, we define the relative trend as the current parameter uncertainty relative to the average of the previous δ days as follows:

$$T_t^{(i)} = \frac{u_{P,t}^{(i)}}{\sum_{j=t-\delta}^{t-1} u_{P,j}^{(i)} / \delta} \tag{15}$$

where $u_{P,j}^{(i)}$ designates the parameter uncertainty of asset i at time j. We refer to $T_t^{(i)}$ to as the relative risk trend of ith asset at time t. When $T_t^{(i)}$ is greater than 1, the relative risk trend is upward, whereas the trend is downward when it is less than 1. If the trend is upward, it makes sense to invest a small amount of capital because the model's decision is unreliable. As there is no clear criterion for how much parameter uncertainty range should be the level of caution, the relative trend between usual and present uncertainty can be used as a reference for warning. This trend analysis will also be useful in the swift handling of the risky situation because it can represent the fact that investors tend to be insensitive in continuous high-risk situations, whereas they overstate the risk when the risk ascends steeply. Specifically, if the trend is upward, the market would excessively fluctuate with panic selling on inflated fears.

On the contrary, we use the reward distribution estimated from the aleatoric branch following the second intuition to consider market uncertainty. The absolute value of the mean of the distribution indicates the expected profit rate. It is worth investing in asset i if the expected profit rate exceeds market uncertainty, that is, $|\bar{v}_t^{(i)}| - u_{M,t}^{(i)} > 0$, where $\bar{v}_t^{(i)}$ and $u_{M,t}^{(i)}$ denotes the mean of the reward distribution and market uncertainty of asset i at time t, respectively.

To evaluate the investment score of each asset, we define the score $z_t^{(i)}$ by combining the two intuitions as follows:

$$z_t^{(i)} = \left\{ 1 - \sigma\left(T_t^{(i)} - 1 \right) \right\} \times \max\left(0, \left| \bar{v}_t^{(i)} \right| - u_{M,t}^{(i)} \right) \tag{16}$$

where $\sigma(\cdot)$ is the sigmoid function. The process of calculating investment scores is presented in Fig. 4 with a concrete example.

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.

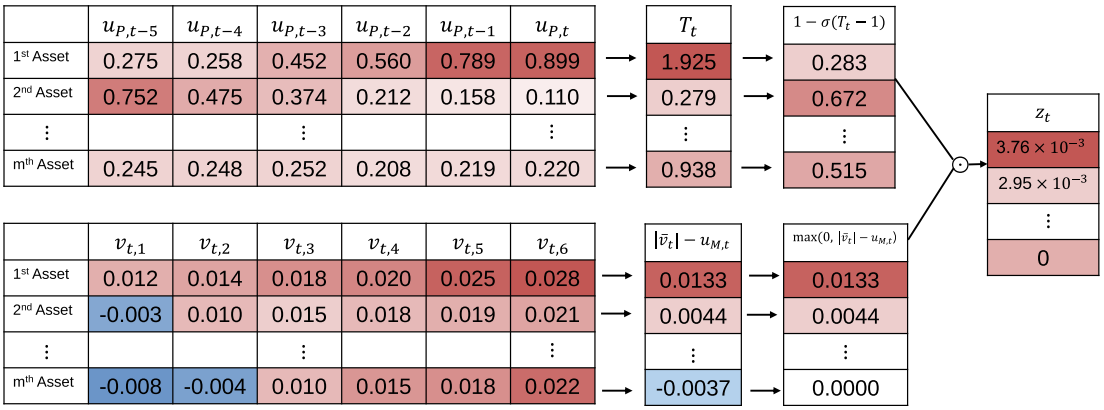


Fig. 4. Example of RAPG where $\delta = 5$ and the number of quantiles $N = 6$. $v_{t,j}$ refers to the jth quantile at time t.

Finally, the portfolio ratio of each asset is calculated by normalizing the scores except for the risk-free asset as follows:

$$b_t^{(i)} = \left(1 - b_t^{(0)}\right) \times \frac{z_t^{(i)}}{\sum_{j=1}^{m} z_t^{(j)}} \tag{17}$$

where $b_t^{(0)}$ indicates the portfolio ratio of the risk-free asset. We do not utilize the softmax function popularly used in previous studies [4], [43] because normalizing with the softmax function leads to large investments in fewer assets and thus not sufficiently hedge the risk.

The portfolio ratio of the risk-free asset is calculated separately as

$$b_t^{(0)} = \max\left(0, \bar{T}_t - \lambda\right) \tag{18}$$

where λ is an adaptable hyperparameter according to the user's risk appetite, and $\bar{T}_t$ refers to the average risk trend over the total asset

$$\bar{T}_t = \frac{\bar{u}_{P,t}}{\sum_{j=t-\delta}^{t-1} \bar{u}_{P,j}/\delta} \tag{19}$$

where $\bar{u}_{P,j}$ refers to the mean of the parameter uncertainty of each asset at time j, which is defined as $\bar{u}_{P,j} = (\sum_{i=1}^{m} u_{P,j}^{(i)}/m)$. Through this risk-free investment ratio, investment in risk assets is reduced when the average risk is upward. This is similar to hedging risks by increasing cash holdings when uncertainties across the market increase due to system risks. As λ increases, the generator creates a higher risk-taking portfolio.

C. Perspective of Multiagent System

Multiagent system (MAS) is a loosely coupled network of several agents that communicate and interact with each other to achieve a specific goal [44], [45]. Largely, MAS is divided into three categories according to their tasks: 1) fully competitive; 2) fully cooperative; and 3) a mix of the two. A competitive system is that each agent competes to achieve their individual goals. On the other hand, in a cooperative system, each agent uses the knowledge of other agents and works together to achieve a common goal. In this respect, our RSMAN is a cooperative system because several independent agents work together to make a portfolio as the common goal of increasing returns and decreasing risks.

Meanwhile, the field of multiagent RL (MARL) is growing rapidly and various approaches that address its challenges have been proposed over the last years [46]–[50]. As defined in [51], MARL has five common approaches: 1) independent learners [52]–[55]; 2) fully observable critic [46], [56]; 3) value function factorization [50], [57]; 4) consensus [58], [59]; and 5) learn to communicate [60], [61]. Among them, RSMAN is similar to the approach of independent learners, in which each agent independently learns its own policy, treating other agents as part of the environment. This idea, called independent Q-learning (IQL), was formalized in [52] for the first time that extends the Q-learning algorithm to the multiagent problem. In a similar way, our RSAs do not share information with other agents and consider them as a part of the environment during the training period. On the other hand, one major difference between our method and IQL is that RSMAN has RAPG that acts as a coordinator to allocate resources according to the expected returns and risks measured by each agent.

V. Experiments

In this section, we evaluate our framework in the U.S. and Korean stock markets to assure validity in the real market. First, we compare our framework with popular baselines. Second, we demonstrate that our framework can generate a portfolio consistent with risk preference λ without the need for retraining. Third, we prove the superiority of RSMAN through comparison with a Bayesian-based uncertainty estimation method [62]. Fourth, we conduct extensive experiments to assess the effects of assigning different weights to $L_{\text{aleatoric}}$ and $L_{\text{epistemic}}$ in 14. Fifth, we conduct a performance comparison experiment between portfolio generators in the form of DNNs to validate the superiority of RAPG.

A. Experimental Settings

1) Datasets: We collect datasets from both markets to demonstrate that our proposed methods operate consistently across a variety of environments. These markets include a

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.

variety of market conditions, such as bulls,[1] bears,[2] and flat[3] markets in the past few decades.

- *U.S. Markets:* The NYSE and NASDAQ datasets are obtained from the Google Finance API.[4] We divide the dataset into the training/test period, that is, from the listing date up to December 2009 as the training period and between January 2010 and May 2020 as the test period. To ensure a sufficient training period, we choose only assets whose listing date was before January 2000. We also use only assets where the capitalization is more than five billion dollars on the last date of training so as to exclude extremely unstable assets; this is because a small cap stock could be manipulated by a host of people. As a result, the number of risk assets is 94.
- *Korean Market:* For the Korean market, we collect datasets of stocks listed on the KOSPI. The KOSPI datasets were acquired from DAISHIN Company Ltd API.[5] The period of our datasets was from the listing date to May 2020. The training/test set is divided by the same standard as the U.S. market. The criteria for selecting assets in the Korean and U.S. markets are the same, except that the market capitalization must be at least 1 trillion won (monetary unit of Korea). We choose 63 risk assets based on these criteria.

2) Evaluation Metrics: We introduce widely used metrics for evaluating portfolio performance. As the investment goal is subjective depending on the user's risk appetite, it is difficult to say what is a better portfolio by evaluating a specific metric. Therefore, several evaluation metrics should be considered comprehensively.

- *Annualized Profit Rate (APR):* First, we can evaluate the profitability of the performance of the portfolio with APR as follows:

$$\mathrm{APR}_T = \left(\frac{\mathrm{CW}_T}{\mathrm{CW}_0}\right)^{\frac{1}{N_Y}} - 1 \tag{20}$$

  where N_Y denotes the number of years in the trading period, CW_T is the cumulative wealth on the last trading day and CW_0 denotes the initial capital. CW_0 is set to 1 for simplicity.
- *Annualized Sharpe Ratio (ASR) [23]:* We also evaluate a portfolio with the ASR to assess the risk-adjusted performance of a portfolio. It is calculated as

$$\mathrm{ASR}_T = \frac{\mathrm{APR}_T}{\mathrm{AV}_T} \tag{21}$$

  where AV_T is the annualized variance of profit rate

$$\mathrm{AV}_T = \sqrt{D_Y} \times \sqrt{\frac{\sum_{t=1}^{T}\left(R_t - \overline{R}_t\right)^2}{T-1}} \tag{22}$$

  where D_Y denotes the number of trading days in a year and $R_t = \mathrm{CW}_t/\mathrm{CW}_{t-1}$ is the profit rate at time t.
- *Maximum DrawDown (MDD) [63]:* The downside risk of a portfolio is an important factor in evaluating the stability of a portfolio. The MDD metric is primarily used to assess it. It is defined as the most significant drawdown measuring the decline from a historical peak and is calculated as

$$\mathrm{MDD}_T = \max_{\gamma \in (0,T)}\left[1 - \frac{\mathrm{CW}_\gamma}{\max_{t\in(0,\gamma)}\mathrm{CW}_t}\right]. \tag{23}$$

- *Calmar Ratio (CR) [24]:* It is a type of risk-adjusted profit rate and is defined as

$$\mathrm{CR}_T = \frac{\mathrm{APR}_T}{\mathrm{MDD}_T}. \tag{24}$$

3) Baselines: To compare the proposed framework's performance with existing strategies, we use the following popular investment strategies as baselines.

- *Equally Weighted Portfolio (EW) [1]:* This is the most traditional strategy that designates the same weight to each asset and holds them until time T. It is closely related to the growth rate of the market. Thus, it is usually considered as a benchmark.
- *Time Series Momentum (TSM) [64]:* Under the assumption that assets with good performance in the past will continue to perform well, this strategy generates a portfolio in proportion to the growth rate of the asset over a certain period.
- *Minimum-Variance Portfolio (MVP) [65]:* MVP minimizes the portfolio variance using the variance of past prices and its associated correlation.
- *On-Line Moving Average Reversion (OLMAR) [66]:* This is a method that exploits a multiple-period mean reversion to overcome the single-period mean reversion assumption.
- *Robust Median Reversion Portfolio (RMR) [67]:* RMR is the online portfolio strategy that exploits mean reversion by an L1-median estimator similar to OLMAR.
- *Portfolio Strategy Based on Deep Q-Networks:* We train Deep Q-Networks (DQN) [68] to predict the same reward defined in this study for a comparative evaluation with deep learning-based methodology. This is a model that is unaware of the uncertainty of the decisions.

We also conduct ablation experiments to investigate the effectiveness of each uncertainty information and mutual distance as follows.

- *RSMAN (PU):* This variant invests proportionally to the scale of parameter uncertainty trend without considering market uncertainty.
- *RSMAN (MU):* This is a variant of RSMAN that generates a portfolio proportionally to the scale of market uncertainty without considering parameter uncertainty.
- *RSMAN (w/o MD):* To prove the superiority of our proposed mutual distance method through a comparison with the existing unsupervised learning-based parameter uncertainty estimation, this is a variant in which the epistemic branch learns a function that maps to a specific point.

[1]A bull market means the condition of a market in which prices are rising consistently.

[2]A bear market refers to a market environment in which stock prices steadily decline for a long time.

[3]A flat market refers to a market that is not a clear upward or downward trend.

[4]https://www.google.com/finance

[5]https://www.daishin.com/

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.

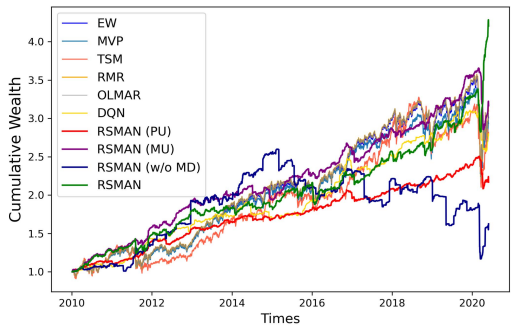


Fig. 5. Performance of RSMAN and baselines. (a) Cumulative wealth in the U.S. market. (b) Cumulative wealth in the Korean market.

4) Implementation Details: In the experiment, we assume that there is no transaction fee because it is negligibly small and is different across to markets. The rate of return versus transaction fee is covered in Section V-B5. The window size l of the state is set to 512, indicating that the model decides by looking at roughly two years of market trends. The number of quantiles N is set to 200, and the output dimension of each fully connected layer of the shared network is 256; thus, the dimension d of the high-level representation ϕ is 256. We set the output dimension of the mapping functions of the epistemic branch to 100. δ for obtaining the uncertainty trend was set to 14. We set μ to 0.0276%, $(1+\mu)^{255}-1 \approx 7.3\%$ because the stock rose by 7% on average during the training period. We use Adam [69] as an optimizer with a learning rate of 0.0005. The networks are trained until the average accuracy of the actions on the training set converges to 95%, and it usually converges within 3000 epochs.

B. Experimental Results

1) Results on the U.S. Markets: Fig. 5(a) shows the cumulative wealth of the RSMAN and baselines. In this experiment, λ was set to 1.3. As the NASDAQ and NYSE markets have been significant bull markets over the past decade, most strategies show superior profitability. Nonetheless, it shows that RSMAN's cumulative wealth is much better than baselines. Looking at the cumulative wealth of RSMAN over time, it is steadily rising to the right without a significant decline. In Table I, we show the performances in terms of quantitative metrics. The metrics (APR, ASR, and CR) indicate better performance as the value increases, whereas the lower value of MDD indicates a more stable performance. Looking at MDD in Table I, the remarkable aspect is that the cumulative wealth of RSMAN fell by only 16.5% during the COVID-19 pandemic, even though the U.S. markets dropped by approximately 38%. This result reveals the effectiveness of our methods under unforeseeable circumstances.

TABLE I
QUANTITATIVE PERFORMANCE ON THE U.S. MARKETS

Method	APR	ASR	MDD	CR
EW	0.110	0.263	0.384	0.287
MVP	0.107	0.273	0.370	0.290
TSM	0.103	0.261	0.358	0.289
OLMAR	0.113	0.265	0.380	0.298
RMR	0.113	0.265	0.380	0.298
DQN	0.102	0.618	0.174	0.568
RSMAN (PU)	0.076	**0.793**	0.168	0.453
RSMAN (MU)	0.116	0.663	0.219	0.528
RSMAN (w/o MD)	0.047	0.137	0.550	0.085
RSMAN	**0.146**	0.641	**0.165**	**0.885**

An interesting observation from the ablation study is that the performance of RSMAN (PU) is much better than that of RSMAN (MU) in terms of stability but is much lower in terms of profitability. The reason for this result is that RSAs are aware of the inherent risks of their decision via parameter uncertainty. Thus, they stick to stable investments in situations familiar to agents. However, it is inferior to all baselines in terms of APR due to the excessive pursuit of stability.

On the contrary, RSMAN (MU) demonstrates superior performance before the market collapse in response to the COVID-19 pandemic but has not recovered subsequently, as shown in Fig. 5(a). This shows that RSMAN (MU) can analyze the volatility of the market in general market conditions with QR, having a good understanding of the period that can make a profit. However, the fact that RSMAN (MU) could not recover the loss from the crash indicates that it cannot prevent making uncertain decisions about outliers.

The comparison between RSMAN and the ablated models supports the argument that the two types of uncertainty should be simultaneously estimated to complement each other. In addition, RSMAN (w/o MD) fails to defeat the baselines in contrast to RSMAN, revealing that the proposed mutual distance efficiently determines the optimal point.

We also examine two types of uncertainty under big events to better understand the behaviors of RSMAN. Fig. 6 shows the average of the parameter uncertainty and market uncertainty of each RSA during the times of crisis. It is interesting to note that the uncertainties during the events are significantly higher than the usual ones. As stock prices generally fluctuate around $1 \sim 3\%$, it is intuitive that the scale of market uncertainty is relatively small. Interestingly, even if the financial crisis of 2007–2009 occurred during the training period, RSMAN estimates high uncertainties.

2) Results on the Korean Markets: Having repeatedly fluctuated and moved sideways over the past decades, the Korean market is appropriate to validate the effectiveness of the proposed methods under different conditions. We also set λ to

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.

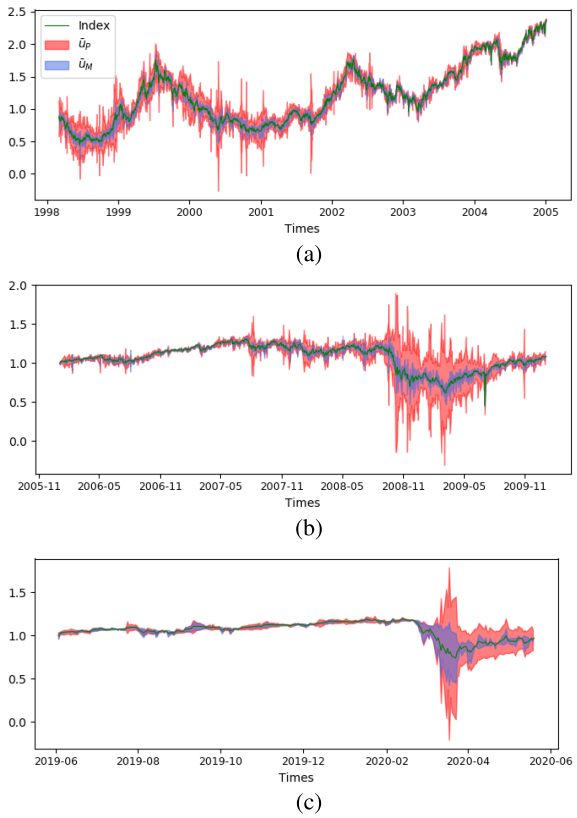


Fig. 6. Uncertainty during big events. (a) Asian financial crisis of 1998–2000 in the Korean market. (b) Financial crisis of 2007–2009 in the U.S. market. (c) COVID-19 pandemic in the U.S. market.

1.3. Fig. 5(b) and Table II show the results of the evaluation of the Korean market. As shown in Fig. 5(b), all variants except for RSMAN (w/o MD) surpass the performances of the baselines. These results show that our framework works effectively in a variety of market environments.

The superior performance of RSMAN compared to the ablated models is evidence supporting the fact that understanding the two types of uncertainty plays a vital role in PM, and the proposed mutual distance is an efficient parameter uncertainty measurement. In addition, Fig. 6(a) shows the estimated uncertainty under the Asian financial crisis. The Korean market experienced an unprecedented fall in the stock market during the crisis and thus expresses high uncertainty despite being a training section. These expressions of uncertainty help the model detect risk and make stable investments during the occurrence of such significant falls.

3) Portfolio Fitting the User's Risk Appetite: One of the great strengths of our framework is that it can create a portfolio that fits the user's risk appetite without retraining. To demonstrate this ability, we perform an experiment on the U.S. market with the risk-related hyperparameter λ changing between $\{0.8, 1, 1.3, 2\}$. A higher λ indicates more risk and more aggressive investment. Fig. 7 shows the results of the experiments. The performances and average risk-free asset rates corresponding to λ are listed in Table III. Consistent with our intuition, we here show that the average rate of risk-free assets in a portfolio increases as λ decreases, indicating

TABLE II

QUANTITATIVE PERFORMANCE ON THE KOREAN MARKET

Method	APR	ASR	MDD	CR
EW	0.037	0.146	0.503	0.074
MVP	0.045	0.206	0.430	0.104
TSM	0.008	0.027	0.631	0.013
OLMAR	0.046	0.170	0.497	0.093
RMR	0.041	0.152	0.505	0.080
DQN	0.071	0.385	0.227	0.315
RSMAN (PU)	0.088	**0.758**	0.174	0.508
RSMAN (MU)	0.116	0.593	**0.161**	0.718
RSMAN (w/o MD)	0.077	0.445	0.329	0.234
RSMAN	**0.139**	0.639	0.194	**0.719**

TABLE III

PERFORMANCE BY THE QUANTITATIVE METRICS AND THE AVERAGE OF RISK-FREE ASSET RATE ACCORDING TO λ

λ	APR	ASR	MDD	CR	Avg. $b^{(0)}$
0.8	0.128	0.663	**0.145**	0.884	0.254
1	0.142	**0.696**	0.165	0.860	0.147
1.3	**0.146**	0.641	0.165	**0.885**	0.067
2	0.127	0.441	0.325	0.391	0.016

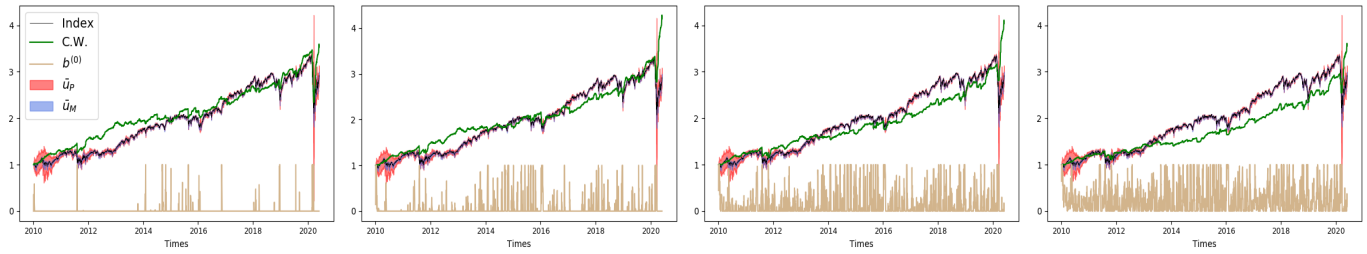
risk-averse investments. Impressively, when λ is 2, it shows the lowest performance, indicating that the strongly risk-taking portfolio is more likely to incur losses. The performances at $\lambda = \{1, 1.3\}$ show that proper hedging has a more positive impact on profitability. When λ is 0.8, it indicates a strongly risk-averse type, so the indicator of stability is high but its profitability is low.

4) Parameter Sensitivity of RAPG: RAPG has a hyperparameter δ to calculate the trend of uncertainty. Here, we evaluate the sensitivity of RAPG to δ. Fig. 8 shows the parameter sensitivity of RAPG. As shown in Fig. 8, the performance of RAPG is stable when δ is between 14 and 21. These results suggest that a miniscule δ is sensitive to outliers; therefore, RAPG makes a hasty decision causing loss. On the contrary, an excessively large δ makes RAPG insensitive to uncertainties.

5) Effect of Transaction Cost: In a real-life setting, transaction cost occurs when buying or selling an asset. In order to consider the transaction cost of various market environments, we investigate the rate of return when the cost is from 0% to 0.6% of the transaction amount. When considering that the trading taxes of the Korean market and the U.S. market are approximately 0.2% and 0.003% except for the capital gains tax, Fig. 9 indicates that RSMAN effectively makes profits even if the transaction cost is reflected.

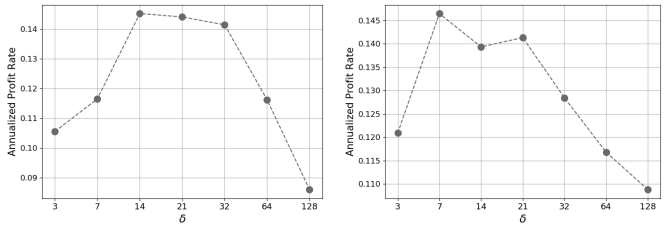
6) Comparison With an Uncertainty Quantification Method: To compare the performance of the proposed framework with an existing uncertainty measurement strategy, we use Bayesian deep learning (BDL) [62] as a baseline. We chose BDL as it can measure aleatoric (market) and epistemic (parameter) uncertainties in a single model. This method measures epistemic uncertainty through Monte Carlo dropout [25], which can be implemented by applying dropout after each fully connected layer of RSA. Following [25], we take 50 Monte Carlo samples with 0.2 and 0.1 dropout rate. To model aleatoric uncertainty, BDL constructs the model in terms of both

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.



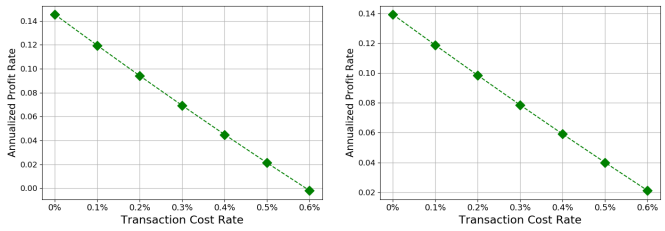
(a) (b) (c) (d)

Fig. 7. Performance of RSMAN according to λ on the U.S. markets. We provide the comparison among the market index (black), cumulative wealth (green), portfolio rate of the risk-free asset (tan), average market uncertainty (blue), and average parameter uncertainty (red). (a) $\lambda = 2$. (b) $\lambda = 1.3$. (c) $\lambda = 1$. (d) $\lambda = 0.8$.



(a) (b)

Fig. 8. Parameter sensitivity to δ. (a) Parameter sensitivity of RAPG in the U.S. markets. (b) Parameter sensitivity of RAPG in the Korean market.



(a) (b)

Fig. 9. Performance of RSMAN according to transaction cost. (a) Annualized profit rate proportional to transaction cost in the U.S. markets. (b) Annualized profit rate proportional to transaction cost in the Korean market.

predictive mean and predictive variance such as $[\hat{\mathbf{r}}_t, \hat{\sigma}_t^2] = f(\mathbf{s}_t)$, where $f(\cdot)$ is a Bayesian neural network. This model is learned by minimizing loss as follows:

$$L(\theta) = \frac{1}{T-l+1} \sum_{t=l}^{T} 2\hat{\sigma}_t{}^{-2} \|r_t - \hat{r}_t\|^2 + \frac{1}{2} \log \hat{\sigma}_t^2. \quad (25)$$

For a fair comparison, we set the hidden size, the learning rate, the window size, the epoch, and the optimizer that are mentioned in the implementation details. The comparison results are shown in Fig. 10 and clearly shows that the proposed method is superior to the existing uncertainty quantification method. This indicates that inferring a reward in distribution form from an alternative branch has richer expressive power than simply predicting a reward in scalar form.

We also evaluate the inference speed on a single V100 GPU (32 GB VRAM) and an Intel Xeon at 2.2 GHz. As a result, the inference time of our method is 0.187 s, whereas BDL

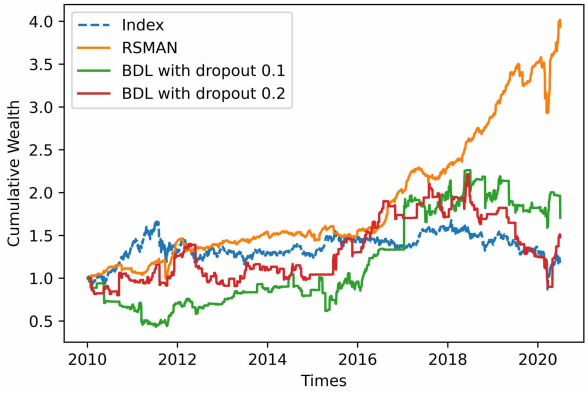


Fig. 10. Comparison between BDL and our method in the Korean market.

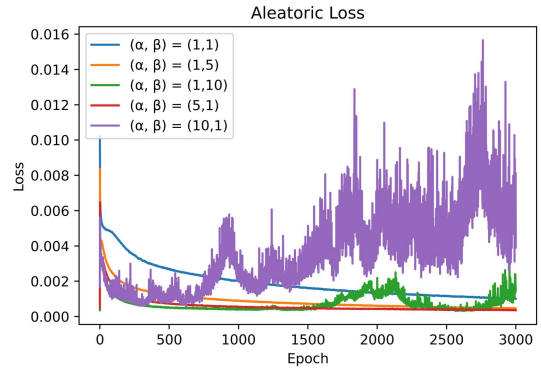
is 1.477 s that is about 7.9 times slower than our method. It suggests that our method also has strengths in terms of scalability, such as high-frequency trading and the expansion of the number of investment assets.

7) Impact of Assigning Different Weights to $L_{aleatoric}$ and $L_{epistemic}$: The loss in (14) for learning each agent is in the form of a multiobjective function that is a combination of two losses. In other words, it is possible that two losses have different ranges of scales, which can affect the learning process such as convergence rates and portfolio profits. Thus, we provide a study to validate the impact of assigning different weights to two losses.

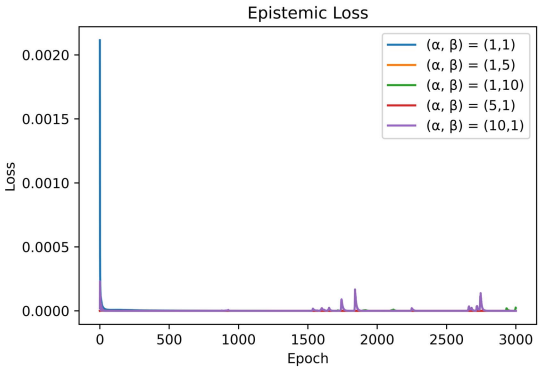
We evaluate the models trained with $(\alpha, \beta) \in [(1, 1), (1, 5), (1, 10), (5, 1), (10, 1)]$. Fig. 11 shows the loss curve according to each multiobjective weight. As can be seen, the scale of the aleatoric loss is approximately ten times larger than that of the epistemic loss. In the case of the aleatoric loss, the loss curve is unstable when (α, β) is $(1, 10)$ and $(10, 1)$. On the other hand, the epistemic loss converges relatively quickly in the early epochs, and the loss curves are stable except for cases when $(\alpha, \beta) = (10, 1)$.

Meanwhile, in Fig. 12, we can observe that the best performance is produced by $(\alpha, \beta) = (1, 5)$. This result suggests that the performance can be improved by elaborately adjusting each weight of the loss rather than simply adding two losses as an equal contribution. To do this, we can apply methods to find a Pareto optimal solution for deep learning [70], [71].

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.



(a)



(b)

Fig. 11. Loss curves for each multiobjective weight. (a) Aleatoric loss curve. (b) Epistemic loss curve.

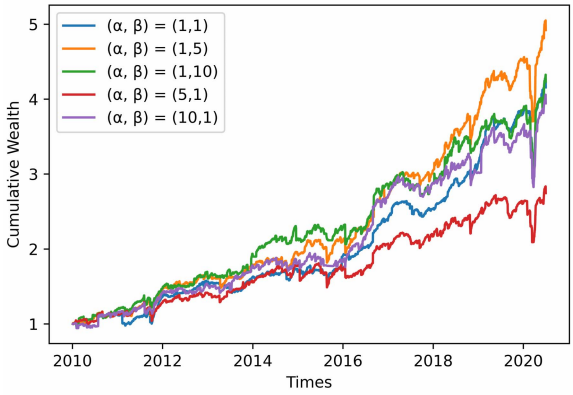


Fig. 12. Cumulative wealth according to each multiobjective weight.

However, each Pareto front needs to learn one new network; thus, it is prohibitive when increasing the number of learnable parameters. In particular, in the case of RSMAN, each asset requires an individual deep network called RSA, and thus, the computational burden increases significantly as the number of assets increases. Therefore, it will be an important future work to study the multiobjective optimization method with a low computational burden suitable to RSMAN.

8) Comparison Between RAPG and a Portfolio Generator in the Form of DNNs: We now conduct the comparative experiment between portfolio generators using a form of deep learning and the proposed RAPG. To this end, we train a portfolio generator in which three fully connected layers are sequentially connected. Let this portfolio generator be referred to as $g(X)$ and X be an $m \times (N+1)$ matrix that consists of N quantiles and u_P estimated from each RSA as follows:

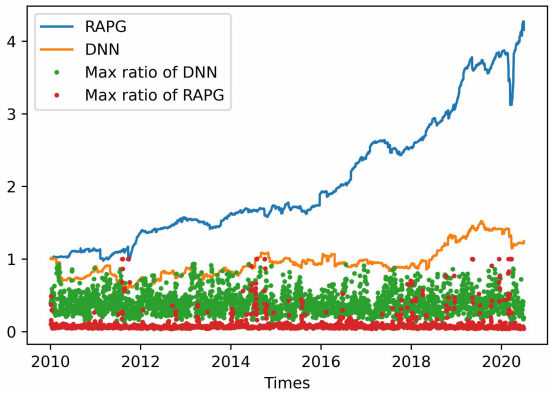


Fig. 13. Comparison result between RAPG and DNN portfolio generator. The blue and orange lines represent the cumulative wealth of RAPG and DNN generator, respectively. The red and green dots are the maximum investment ratio for each time of RAPG and DNN generator, respectively.

$$X = \begin{bmatrix} v_1^{(1)} & v_2^{(1)} & \dots & v_N^{(1)} & u_P^{(1)} \\ v_1^{(2)} & v_2^{(2)} & \dots & v_N^{(2)} & u_P^{(2)} \\ \vdots & & \ddots & \vdots & \vdots \\ v_1^{(m)} & v_1^{(m)} & \dots & v_N^{(m)} & u_P^{(m)} \end{bmatrix}. \tag{26}$$

In the first and second layers, we use the ReLU as an activation function, and in the last layer, we use the hyperbolic tangent function to output the investment ratio between -1 and 1. We train the generator by maximizing daily profits with a gradient ascent method over 10 000 epochs.

As shown in Fig. 13, the return of RAPG is superior to DNN. The red and green dots refer to the maximum investment ratio for each time of RAPG and DNN generators, respectively, which shows that RAPG does not invest much in an asset except for a few times, while DNN mostly invests in specific assets at a high rate. In other words, it means that the DNN generator does not sufficiently distribute the risk. On the other hand, RAPG can efficiently cope with risks by diversification.

VI. Conclusion

In this article, we proposed a novel PM framework called RSMAN to consider the two types of uncertainty—*market uncertainty* and *parameter uncertainty*. To estimate the two types of uncertainty, we successfully applied QR and mutual distance. In addition, we developed a new portfolio generator, RAPG, which appropriately utilizes the estimated uncertainties to cope with dangerous situations. We demonstrated that RSMAN could not only create a robust portfolio to risky situations but also adapt the characteristics of a portfolio according to the user's risk appetite without any need for retraining. Extensive experimental results on the U.S. and Korean markets showed that RSMAN is valid in various markets such as bull, bear, and flat markets. There are several interesting future ideas that we hope to explore. First, in our study, the architecture

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.

comprises only a simple fully connected layer. In the future, we would like to deploy more advanced modules, such as attention. Second, our method can be combined with the state-of-the-art value-based RL algorithms. Therefore, it would be more meaningful to confirm performance improvements by combining our uncertainty quantification method with other RL approaches. Third, we did not consider the correlation between RSAs, but it is known that considering the correlation is an integral factor in the modern portfolio theory. The theory argues that rational investors reduce risks by investing their assets distributively in low correlation stocks. In other words, for more sophisticated portfolio generation, it is a necessary consideration in future work. Fourth, in the future, we could try applying RSMAN to the cryptocurrency market.

REFERENCES

[1] V. DeMiguel, L. Garlappi, F. J. Nogales, and R. Uppal, "A generalized approach to portfolio optimization: Improving performance by constraining portfolio norms," *Manage. Sci.*, vol. 55, no. 5, pp. 798–812, May 2009.

[2] H. Konno and H. Yamazaki, "Mean-absolute deviation portfolio optimization model and its applications to Tokyo stock market," *Manag. Sci.*, vol. 37, no. 5, pp. 519–531, 1991.

[3] H. Markowitz, "Portfolio selection," *J. Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[4] J. Wang, Y. Zhang, K. Tang, J. Wu, and Z. Xiong, "AlphaStock: A buying-winners-and-selling-losers investment strategy using interpretable deep reinforcement attention networks," in *Proc. 25th Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 1900–1908.

[5] G. Tsang, J. Deng, and X. Xie, "Recurrent neural networks for financial time-series modelling," in *Proc. 24th Int. Conf. Pattern Recognit. (ICPR)*, Aug. 2018, pp. 892–897.

[6] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, "Deep direct reinforcement learning for financial signal representation and trading," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 3, pp. 653–664, Mar. 2016.

[7] O. Mihatsch and R. Neuneier, "Risk-sensitive reinforcement learning," *Mach. Learn.*, vol. 49, no. 2, pp. 267–290, 2002.

[8] S. Mannor and J. Tsitsiklis, "Mean-variance optimization in Markov decision processes," 2011, *arXiv:1104.5601*.

[9] R. Koenker and K. Hallock, "Quantile regression," *J. Econ. Perspect.*, vol. 15, no. 4, pp. 143–156, 2001.

[10] L. Ruff *et al.*, "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.

[11] J. Moody and M. Saffell, "Learning to trade via direct reinforcement," *IEEE Trans. Neural. Netw.*, vol. 12, no. 4, pp. 875–889, Jul. 2001.

[12] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.

[13] W. Bao, J. Yue, and Y. Rao, "A deep learning framework for financial time series using stacked autoencoders and long-short term memory," *PLoS ONE*, vol. 12, no. 7, Jul. 2017, Art. no. e0180944.

[14] F. Feng, H. Chen, X. He, J. Ding, M. Sun, and T.-S. Chua, "Enhancing stock movement prediction with adversarial training," 2018, *arXiv:1810.09936*.

[15] Z.-R. Lai, D.-Q. Dai, C.-X. Ren, and K.-K. Huang, "A peak price tracking-based learning system for portfolio selection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 7, pp. 2823–2832, Jul. 2017.

[16] Z.-R. Lai, D.-Q. Dai, C.-X. Ren, and K.-K. Huang, "Radial basis functions with adaptive input and composite trend representation for portfolio selection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 12, pp. 6214–6226, Dec. 2018.

[17] Y. Ye *et al.*, "Reinforcement-learning based portfolio management with augmented asset movement prediction states," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 1, pp. 1112–1119.

[18] A. M. Aboussalah and C.-G. Lee, "Continuous control with stacked deep dynamic recurrent reinforcement learning for portfolio optimization," *Expert Syst. Appl.*, vol. 140, Feb. 2020, Art. no. 112891.

[19] Y. Ding, W. Liu, J. Bian, D. Zhang, and T.-Y. Liu, "Investor-imitator: A framework for trading knowledge extraction," in *Proc. 24th Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 1310–1319.

[20] L. Bisi, L. Sabbioni, E. Vittori, M. Papini, and M. Restelli, "Risk-averse trust region optimization for reward-volatility reduction," 2019, *arXiv:1912.03193*.

[21] N. Vadori, S. Ganesh, P. Reddy, and M. Veloso, "Risk-sensitive reinforcement learning: A Martingale approach to reward uncertainty," 2020, *arXiv:2006.12686*.

[22] W. Dabney, G. Ostrovski, D. Silver, and R. Munos, "Implicit quantile networks for distributional reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1096–1105.

[23] W. F. Sharpe, "The Sharpe ratio," *J. Portfolio Manage.*, vol. 21, no. 1, pp. 49–58, 1994.

[24] T. W. Young, "Calmar ratio: A smoother tool," *Futures*, vol. 20, no. 1, p. 40, 1991.

[25] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1050–1059.

[26] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra, "Weight uncertainty in neural networks," 2015, *arXiv:1505.05424*.

[27] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6402–6413.

[28] Y. LeCun. (1998). *The MNIST Database of Handwritten Digits*. [Online]. Available: http://yann.lecun.com/exdb/mnist/

[29] D. J. C. MacKay, "A practical Bayesian framework for backpropagation networks," *Neural Comput.*, vol. 4, no. 3, pp. 448–472, May 1992.

[30] T. Yang *et al.*, "Exploration in deep reinforcement learning: A comprehensive survey," 2021, *arXiv:2109.06668*.

[31] I. Osband, C. Blundell, A. Pritzel, and B. Van Roy, "Deep exploration via bootstrapped DQN," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 4026–4034.

[32] R. Y. Chen, S. Sidor, P. Abbeel, and J. Schulman, "UCB exploration via Q-ensembles," 2017, *arXiv:1706.01502*.

[33] T. M. Moerland, J. Broekens, and C. M. Jonker, "Efficient exploration with double uncertain value networks," 2017, *arXiv:1711.10789*.

[34] Y. Tang and S. Agrawal, "Exploration by distributional reinforcement learning," 2018, *arXiv:1805.01907*.

[35] N. Nikolov, J. Kirschner, F. Berkenkamp, and A. Krause, "Information-directed exploration for deep reinforcement learning," 2018, *arXiv:1812.07544*.

[36] B. Mavrin, H. Yao, L. Kong, K. Wu, and Y. Yu, "Distributional reinforcement learning for efficient exploration," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 4424–4434.

[37] T. L. Lai and H. Robbins, "Asymptotically efficient adaptive allocation rules," *Adv. Appl. Math.*, vol. 6, no. 1, pp. 4–22, Mar. 1985.

[38] A. L. Strehl and M. L. Littman, "A theoretical analysis of model-based interval estimation," in *Proc. 22nd Int. Conf. Mach. Learn. (ICML)*, 2005, pp. 856–863.

[39] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," *Biometrika*, vol. 25, pp. 285–294, Dec. 1933.

[40] D. Silver *et al.*, "Mastering the game of go without human knowledge," *Nature*, vol. 550, pp. 354–359, Oct. 2017.

[41] G. Song and W. Chai, "Collaborative learning for deep neural networks," 2018, *arXiv:1805.11761*.

[42] G. C. Harcourt, P. H. Karmel, and R. H. Wallace, *Economic Activity*, vol. 427. Cambridge, U.K.: CUP Archive, 1967.

[43] Z. Jiang, D. Xu, and J. Liang, "A deep reinforcement learning framework for the financial portfolio management problem," 2017, *arXiv:1706.10059*.

[44] G. Weiss, *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*. Cambridge, MA, USA: MIT Press, 1999.

[45] J. Hao and H.-F. Leung, *Interactions in Multiagent Systems: Fairness, Social Optimality and Individual Rationality*. Cham, Switzerland: Springer, 2016.

[46] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," 2017, *arXiv:1706.02275*.

[47] Y. Yang, R. Luo, M. Li, M. Zhou, W. Zhang, and J. Wang, "Mean field multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5571–5580.

[48] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2961–2970.

[49] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 2974–2982.

[50] T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4295–4304.

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.

[51] A. OroojlooyJadid and D. Hajinezhad, "A review of cooperative multi-agent deep reinforcement learning," 2019, *arXiv:1908.03963*.
[52] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *Proc. Int. Conf. Mach. Learn.*, 1993, pp. 330–337.
[53] T. Fuji, K. Ito, K. Matsumoto, and K. Yano, "Deep multi-agent reinforcement learning using DNN-weight evolution to optimize supply chain performance," in *Proc. 51st Hawaii Int. Conf. Syst. Sci.*, 2018, pp. 1278–1287.
[54] A. Tampuu *et al.*, "Multiagent cooperation and competition with deep reinforcement learning," *PLoS ONE*, vol. 12, no. 4, Apr. 2017, Art. no. e0172395.
[55] J. Foerster *et al.*, "Stabilising experience replay for deep multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1146–1155.
[56] H. Ryu, H. Shin, and J. Park, "Multi-agent actor-critic with generative cooperative policy network," 2018, *arXiv:1810.09206*.
[57] P. Sunehag *et al.*, "Value-decomposition networks for cooperative multi-agent learning," 2017, *arXiv:1706.05296*.
[58] K. Zhang, Z. Yang, H. Liu, T. Zhang, and T. Basar, "Fully decentralized multi-agent reinforcement learning with networked agents," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5872–5881.
[59] L. Cassano, K. Yuan, and A. H. Sayed, "Multiagent fully decentralized value function learning with linear convergence rates," *IEEE Trans. Autom. Control*, vol. 66, no. 4, pp. 1497–1512, Apr. 2020.
[60] J. N. Foerster, Y. M. Assael, N. de Freitas, and S. Whiteson, "Learning to communicate with deep multi-agent reinforcement learning," 2016, *arXiv:1605.06676*.
[61] E. Jorge, M. Kågebäck, F. D. Johansson, and E. Gustavsson, "Learning to play guess who? And inventing a grounded language as a consequence," 2016, *arXiv:1611.03218*.
[62] A. Kendall and Y. Gal, "What uncertainties do we need in Bayesian deep learning for computer vision?" 2017, *arXiv:1703.04977*.
[63] M. Magdon-Ismail and A. F. Atiya, "Maximum drawdown," *Risk Mag.*, vol. 17, no. 10, pp. 99–102, 2004.
[64] T. J. Moskowitz, Y. H. Ooi, and L. H. Pedersen, "Time series momentum," *J. Financial Econ.*, vol. 104, no. 2, pp. 228–250, 2012.
[65] R. Jagannathan and T. Ma, "Risk reduction in large portfolios: Why imposing the wrong constraints helps," *J. Finance*, vol. 58, no. 4, pp. 1651–1683, Aug. 2003.
[66] B. Li and S. C. H. Hoi, "On-line portfolio selection with moving average reversion," 2012, *arXiv:1206.4626*.
[67] D. Huang, J. Zhou, B. Li, S. C. H. Hoi, and S. Zhou, "Robust median reversion strategy for online portfolio selection," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 9, pp. 2480–2493, Sep. 2016.
[68] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.
[69] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[70] X. Lin, H.-L. Zhen, Z. Li, Q.-F. Zhang, and S. Kwong, "Pareto multi-task learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeuIPS)*, vol. 32, 2019, pp. 12060–12070.
[71] D. Mahapatra and V. Rajan, "Multi-task learning with user preferences: Gradient descent with controlled ascent in Pareto optimization," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 6597–6607.



Kidon Park received the B.S. degree in electronic engineering from Kwangwoon University, Seoul, South Korea, in 2019. He is currently pursuing the master's degree with the Department of Brain and Cognitive Engineering, Korea University, Seoul.

His current research interests include artificial intelligence and pattern recognition.



Hong-Gyu Jung received the B.S. degree in electronic engineering and the M.S. degree in information telecommunication engineering from Soongsil University, Seoul, South Korea, in 2012 and 2014, respectively. He is currently pursuing the Ph.D. degree with the Department of Brain and Cognitive Engineering, Korea University, Seoul.

His current research interests include artificial intelligence and pattern recognition.



Tae-San Eom received the B.S. degree in computer engineering from Sejong University, Seoul, South Korea, in 2019. He is currently pursuing the Ph.D. degree with the Department of Brain and Cognitive Engineering, Korea University, Seoul.

His current research interests include artificial intelligence and pattern recognition.



Seong-Whan Lee (Fellow, IEEE) received the B.S. degree in computer science and statistics from Seoul National University, Seoul, South Korea, in 1984, and the M.S. and Ph.D. degrees in computer science from the Korea Advanced Institute of Science and Technology, Seoul, in 1986 and 1989, respectively.

He is currently the Head of the Department of Artificial Intelligence, Korea University, Seoul. His current research interests include artificial intelligence, pattern recognition, and brain engineering.

Dr. Lee is a fellow of the IAPR and the Korea Academy of Science and Technology.

Authorized licensed use limited to: Somaiya University. Downloaded on March 05,2026 at 05:43:38 UTC from IEEE Xplore. Restrictions apply.